Building a Pizzeria Backend: SQL, Schema Design, and Data Wrangling

Following a guided project by Adam Finer, I undertook the "Ben's Pizzeria" scenario: a brief requiring a ground-up database for order tracking, stock control, and staff management.

While the video provided a great architectural roadmap, executing it in my own environment using MySQL and Navicat turned into a massive, hands-on learning experience in data engineering and debugging.

Phase 1: Architecture, ERD Design, and Mock Data

The first step was normalizing the data. Instead of one giant, messy spreadsheet, I built a relational schema separating entities into 10 distinct tables (Customers, Addresses, Orders, Menu, Ingredients, etc.). To visualize and document this structure clearly, I designed a comprehensive **Entity-Relationship Diagram (ERD)**. This diagram mapped out all my Primary and Foreign Keys, ensuring that the relational logic between my core operations (orders, inventory, and staff) was watertight before I wrote a single line of SQL.

Generating the mock data was where things got interesting. I built out CSV files for everything, ensuring my order rows mapped perfectly to valid customer_id and address_id keys. However, when I attempted to bulk-import these CSVs into Navicat, I hit a wall of errors.

I quickly learned the painful nuances of EOL (End of Line) characters. Because of how Excel exports data, Navicat was reading my entire spreadsheet as one single line. By diving into the wizard settings and hardcoding the Record Delimiter to CRLF, I finally got the rows to parse. I also had to write data-cleaning scripts to strip out invisible trailing commas and empty rows that were causing 1062 Duplicate Entry and Primary Key constraint violations.

Phase 2: Tweaking the Schema

Once the data was loaded, I realized my schema didn't perfectly match my reality. For example, my ingredients used alphanumeric codes like ING001, but my database schema was strictly expecting INT values. I had to write an ALTER TABLE script to convert my IDs to VARCHAR(10). Because my tables were already tightly bound by Foreign Keys, I learned how to use SET FOREIGN_KEY_CHECKS = 0; to temporarily uncouple the database, make my structural updates, and lock it back down.

Phase 3: Querying for the Business

The final phase was writing the SQL queries that a BI tool would eventually read. This was an incredible exercise in logic and syntax:

- I used extensive LEFT JOIN operations to unify my orders with my menu and delivery addresses.
- I built a custom view called stock_one using subqueries to track inventory depletion. I actually had to debug this view myself when a 1054 Unknown Column error popped

up—I had successfully fetched the ingredient name in my inner query but forgot to pass it through to the outermost SELECT statement!

- For the staff payroll query, I had to master order-of-operations in SQL. By balancing parentheses inside a massive TIMEDIFF() equation, I accurately calculated shift durations by isolating minutes, converting them to decimal hours, and multiplying by the hourly rate. (I also learned to subtract start from end, rather than end from start, to avoid paying my fictional employees negative money!).

Next Steps

Currently, the backend architecture and logic are complete. I have perfectly validated order transactions flowing through my views. The next step in this project will be connecting this live database to a visualization tool to build out the interactive front-end dashboard!